

WHAT ARE RERANKERS?

Table of contents

- [The role of rerankers in search](#)
- [How do rerankers work?](#)
- [Optimizing RAG with second-stage reranking](#)
- [How to select a reranker model?](#)
- [Reranking with MongoDB and Voyage AI](#)
- [Conclusion](#)
- [FAQs](#)

Rerankers are models trained to evaluate and rank the relevance of retrieved documents against a given query. These types of models have been part of search engine architectures for

built upon search and [large language models \(LLMs\)](#).

The role of rerankers in search

To understand the role and utility of rerankers, we first need to explore how reranking fits in search systems.

Search exists in various forms, including:

- **Full-text search**, such as keyword-based methods like BM25.
- **Semantic search**, which finds relevant content based on meaning.
- **Hybrid search**, which combines both full-text and semantic approaches.

Full-text search

Full-text search allows for searching through entire document collections by matching queries against document content, such as keyword-based matching. It uses algorithms like BM25, which rank documents based on term frequency and inverse document frequency, ensuring that more relevant documents appear higher in search results. While effective for structured and precise queries, keyword-based search struggles with synonyms, context, and intent. Modern search engines typically combine these core techniques with features like fuzzy matching for typos and semantic analysis for understanding related concepts.

Embedding-based semantic search

Semantic search aims to improve search accuracy by understanding the contextual meaning of data rather than just matching keywords. It leverages natural language processing (NLP) techniques to understand the intent behind a query and the context within the content.

A common implementation of semantic search involves using vector representations, known as **embeddings**, to convert queries and documents into a semantic and high-dimensional vector space. These embeddings are generated using an embedding model and stored in a vector database. By performing a **vector search** between the query's embedding and the stored document embeddings, a similarity measure can be calculated to identify the most relevant documents to the query. This approach is

known as dense retrieval.

Vector search is fast and efficient

Vector search excels at document retrieval due to its speed and efficiency in finding semantically relevant documents. This is crucial when working with massive databases that scale to billions of documents. The speed is due to the fact that data has already been converted to vector embeddings and stored in a **vector database** such as **MongoDB Atlas**. As a result, the search process consists of low-complexity vector comparisons, such as cosine similarity.

Optimized for speed

This speed comes with a trade-off: some meaning is lost when compressing data into vector embeddings. As a result, vector search delivers fast results, but they may not be in perfect relevance order, and some may not reflect the right level of similarity. This is where rerankers come in. They improve search results by reranking and selecting the best candidates to use in the downstream process.

How do rerankers work?

Rerankers are models that assign relevance scores to a query and retrieved documents. As the name suggests, they refine the ranking of documents initially retrieved using embedding-based methods like semantic search. By scoring documents based on relevance, rerankers improve retrieval accuracy by selecting the most relevant subset of the initially retrieved documents.

Rerankers are cross-encoders

Modern rerankers are transformer-based models referred to as cross-encoders. These models jointly encode the query and each document together, allowing them to capture deeper

relationships and context. Cross-encoders apply self-attention, a mechanism in transformer models that weighs the importance of different words in an input relative to each other. By processing the combined query-document input, self-attention enables the model to focus on the most relevant words in context. As a result, rerankers better grasp the search query's intent in relation to the document's content. They also navigate subtle nuances and ambiguities that can significantly affect the relevance and accuracy of retrieval results.

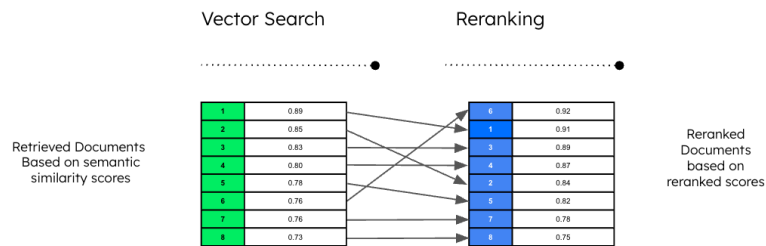
Embedding models are bi-encoders, which means they encode the query and the documents separately, leading to less precision and missing contextual nuances. This is why rerankers are more effective than embedding models for high-precision retrieval.

Trade-offs of rerankers

Rerankers are computationally expensive. Unlike embedding models that precompute and store embeddings for efficient comparison, rerankers process each query-document pair through the reranking model at query time. Cross-encoding every query-document pair grows linearly with the number of candidates, and the quadratic nature of self-attention in transformers further contributes to the computational burden. This makes them slower and more resource-intensive, hence not practical when dealing with a large number of documents.

Two-stage retrieval

Rerankers work best as a second-stage ranking retrieval step. The initial heavy lifting is done by fast and efficient embedding-based vector search, which retrieves a set of documents from an extensive knowledge base but may lack the high precision required in some use cases. Rerankers then refine a set of candidates, providing the most relevant documents to improve the retrieval accuracy.



Reranking after initial vector search

Optimizing RAG with second-stage reranking

Retrieval-augmented generation (RAG) combines document retrieval with large language models (LLMs) to produce more accurate and contextually relevant responses. This technique is widely used in LLM-based chatbots and other intelligent applications to incorporate external knowledge into LLMs. RAG relies on semantic search to find documents relevant to the user's query. These documents are then included in the [prompt](#), enabling the LLM to generate answers grounded in the provided data.

RAG is built upon search

Since RAG relies on semantic search for document retrieval, it inherits the limitations of semantic search's retrieval precision. This might be adequate for some use cases, but optimizing retrieval precision can lead to more accurate responses and fewer hallucinations.

Context window and recall

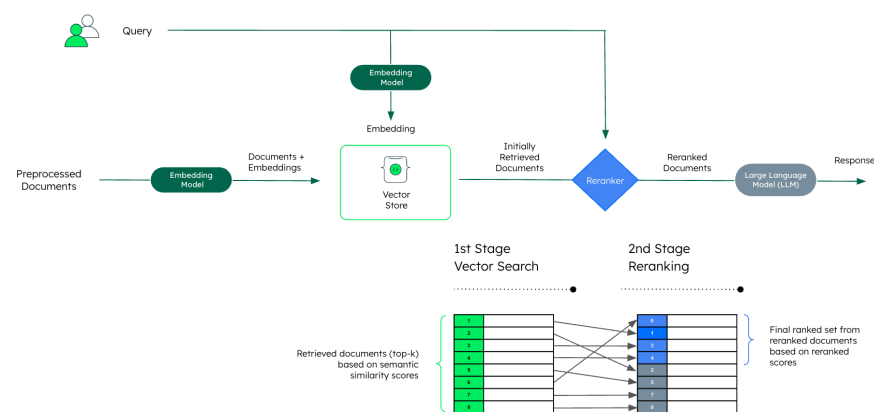
Additionally, when dealing with large knowledge bases, vector search may retrieve numerous documents—too many to fit within an LLM's context window. This can result in missing more relevant documents. Moreover, the performance of large language models

tends to decline when too much information is included in the prompt. Selecting only the most relevant documents positively impacts the resulting accuracy. This concept is often referred to as the LLM's recall, which is its ability to retrieve pertinent information from the provided context.

For these reasons, the basic RAG implementation described here is often called "naive RAG." While it serves as a starting point, real-world production use cases typically require more sophisticated and optimized versions to achieve satisfactory results.

Second-stage reranking in RAG pipelines

A common optimization technique involves using reranking models as a second-stage process to refine the retrieved documents. Only the top results from this reranked set are kept for the final generation, ensuring optimal accuracy in responses.



RAG architecture with two-stage retrieval

In most cases, adding reranking as a second-stage ranking retrieval step after an initial embedding-based semantic search will increase retrieval accuracy and provide better overall results for your RAG solution.

How to select a reranker model?

In the context of dense retrieval for second-stage reranking and retrieval-augmented generation (RAG), the choice of a reranker depends primarily on the latency vs. accuracy trade-off.

As covered previously, rerankers are cross-encoders, meaning they process pairs of queries and documents jointly. This approach significantly improves accuracy but comes at the cost of higher computational requirements, which directly impacts latency.

Different factors directly impact the induced latency by rerankers:

Number of retrieved candidates (top-k)

Since rerankers are computationally expensive, it is important to minimize the number of candidates they process. As a broad indication:

- Vector search should retrieve between 50 and 1,000 candidate documents.
- The reranker should refine this to the top five to 20, ensuring only the most relevant documents are considered.

Please note that these numbers are purely hypothetical and can vary significantly depending on the specific domain and the downstream task. The ideal top-k depends on the initial retrieval quality and the other factors mentioned here. If your initial search

is highly effective, you can afford to pass fewer documents for reranking, reducing latency. A weaker retriever may require a larger candidate set.

Reranker model sizes

Like other types of models, rerankers come in different sizes. The larger the model:

- the better the accuracy (as it captures deeper semantic relationships).
- the higher the latency and computational cost.

Choosing the right size depends on your system's capacity and the acceptable latency for user experience.

Query and document length

The length of both queries and documents affects the choice of reranker:

- for short queries and short documents, a smaller and faster reranker like Voyage AI rerank-2-lite may be sufficient.
- for long queries and long documents, a more powerful reranker like Voyage AI rerank-2 will better capture nuanced relationships.

Additionally, consider chunk size in the vector database. A stronger reranker will be needed to capture fine-grained semantic relevance if you store larger text chunks.

Selecting the right reranker is a balancing act between accuracy, latency, and computational resources. If latency is a concern, opt for lighter models or limit top-k. If accuracy is paramount, invest in a stronger reranker and optimize retrieval quality to reduce the number of candidates it processes.

Reranking with MongoDB and Voyage AI

Search and retrieval-augmented generation (RAG) systems can be optimized in numerous ways beyond just implementing rerankers. MongoDB Atlas Vector Search offers developers advanced query capabilities such as hybrid search, metadata pre-filtering, and vector quantization. However, to achieve even

greater accuracy and performance, integrating reranking is a necessary step.

By leveraging [Voyage AI](#)'s state-of-the-art [embedding models](#) and [rerankers](#) with MongoDB Atlas Vector Search, developers can ensure that their AI-powered applications deliver the most relevant information with high precision and enhanced reliability leading to contextually rich and grounded answers from LLMs. Voyage AI offers a range of embedding models, from small to large, to meet various technical requirements and computational resources. Additionally, domain-specific embedding models are available to capture the nuances of particular fields and industries.

Start building today. Simplify your developer experience and create more value with a fully managed, secure database integrated with a vast [AI partner ecosystem](#). This ecosystem includes all major cloud providers, LLM model providers, and system integrators.

Relevant resources

Explore MongoDB Atlas, the database with built-in search, vector, and more capabilities, [register for free](#) now.

To learn more about Voyage AI, you can read more [here](#).

For strategic advice and implementation support for search and [AI stack](#), visit our [MongoDB AI Applications Program](#) for more details.

Conclusion

Rerankers are a crucial component for enhancing search performance and the quality of document retrieval in retrieval-augmented generation (RAG) systems. They have demonstrated significant improvements, especially in the context of LLM-based

applications, which are sensitive to the amount of information provided and limited by context window size. While vector search is fast and efficient, rerankers offer greater precision at the cost of higher resource usage. This trade-off often leads to the implementation of multi-stage retrieval systems for similarity search and RAG.

However, rerankers are not the only optimization that can significantly improve search and RAG systems. The choice of embedding models, whether general-purpose or domain-specific, and their fine-tuning can also enhance retrieval accuracy. Additionally, the chunking strategy and the use of advanced search capabilities like hybrid search, pre-filtering and prompt compression are important optimizations to consider.

What are the best reranker models?



The best reranking models depend on your search system's specific needs, such as the precision and latency of the initial retrieval process and the latency considerations for the user experience. A larger reranking model will offer best-in-class precision and accuracy but at the cost of more computational resources and increased downstream latency. Conversely, a smaller reranker will be faster but less precise. Many factors need to be taken into account, and

What is the difference between reranking and embedding models?



What is the role of a reranker in a RAG pipeline?



Get started with MongoDB Atlas

Try Free

📍 English

About

Careers

Investor Relations

Legal

Privacy Policy

GitHub

Security Information

Trust Center

Follow Us

Build

MongoDB Atlas

Support

Contact Us

Customer Portal

Atlas Status

Product Updates

Customer Support

Manage Cookies

🔒 Your Privacy Choices

Data Basics

Vector Databases

Enterprise Advanced

Community Edition

Voyage AI

NoSQL Databases

Document Databases

RAG Database

ACID Transactions

MERN Stack

Agent Memory

MEAN Stack

© 2026 MongoDB, Inc.

